We'd like to spend a couple of paragraphs trying to convince you that the lack of static typing is not a problem when it comes to writing reliable applications. In fact, an important part of Ruby style is trusting Ruby's dynamic typing to help you and not hurt you. We're not trying to criticize other languages here. Instead, we'd just like to contrast approaches.

The reality is that the static type systems in most mainstream languages don't help that much in terms of program correctness. If Java's type system were perfectly reliable, for example, it wouldn't need to implement `ClassCastException`. The exception is necessary, though, because there is runtime type uncertainty in Java (as there is in TypeScript, C#, and others). Static typing can be good for optimizing code, it can be useful for communicating intent, and it can help IDEs do clever things with tooltip help, but we haven't seen much evidence that it promotes more reliable code.

Static typing also has costs. Statically typed languages are typically more verbose than dynamic ones, which can cause the business logic to be obscured by the type declaration clutter. It's not unheard of to spend time convincing the type checking system that the thing you know is right is actually also legal.

On the other hand, once you use Ruby for a while, you realize that dynamically typed variables add to your productivity in many ways. You'll also be surprised to discover that your worst fears about the type chaos were unfounded. Large, long-running Ruby programs run significant applications and just don't throw many type-related errors. Why is this?

Partly it's a question of program structure. In many cases the structure of the program makes type errors unlikely even if the language isn't explicitly checking for them. You put `Person` objects in, and few lines later the code sends `Person` objects out. Add in some reasonably meaningful variable names and you're already minimizing the possibility of type errors.

Good style techniques limit the possibility of type errors in Ruby. If you use a variable for some purpose, chances are good you'll be using it for the same purpose when you access it again three lines later. The kind of chaos that *could* happen just doesn't happen.

On top of that, folks who code Ruby a lot tend to adopt a certain style of coding. They write lots of short methods and tend to test as they go along. The short methods mean that the scope of most variables is limited; there just isn't that much time for things to go wrong with their type. The testing catches the silly errors when they happen; typos and the like just don't get a chance to propagate through the code. As an added bonus, short, testable methods tend to have other benefits for code quality in addition to limiting the possibility of type errors.

The upshot is that the "safety" in "type safety" is often illusory and that coding in a more dynamic language such as Ruby is both safe and productive. So, if you're nervous about the lack of static typing in Ruby, we suggest you try to put those concerns on the back burner for a little while and give Ruby a try. We think you'll be surprised how rarely you see errors because of type issues and how much more productive you feel once you start to exploit the power of dynamic typing.

Classes Aren't Types

If you've coded in strongly typed languages, you may have been taught that the *type* of an object is the same as its *class*—all objects are instances of some class, and that class is the object's type. The class defines the operations (methods) the object can support, along with the state (instance variables) on which those methods operate. Let's look at some Java code: